CSC111 Project Topic: A Tool to Analyze Consequences of Road Segment Closure by Graphical Analysis of Ontario's Road Network

Khalid Alshikhmubarak, Elias Mohammed A Alhazzaa, Fayiz Ali

March 30, 2026

Problem Description and Research Question

Transportation networks rely on interconnected highways, roads, bridges, and tunnels. These networks are always vulnerable to construction closures, accidents, and natural disasters, which could affect the structure or even connectivity of the road network. Each year, numerous cities are threatened by natural disasters that could temporarily close down significant routes. When these routes become unusable, people may not have access to fundamental necessities, such as hospitals and food providers. Therefore, it is important to study transportation networks to prevent an area from being isolated from the rest of its city during emergencies, or even if an isolation does not happen, then to understand how deep would the impact of a road segment closure for emergency/construction reasons be on motorists. This predictive analysis would help prepare the province better and take proactive measures to minimize the impacts of road closures. This goal motivates us to develop a tool to deeply analyze road networks: in particular to use technology and computer science to analyze how road closure would impact travel.

The province of Ontario is the most populous province in Canada, and about half of its population lives in the Greater Toronto Area (Ontario Legislative Assembly). Thus, it is imperative that it keeps a check on the impediments people face due to road closures. A week ago, there was a closure on College and Bay Streets in the Downtown of Toronto, which forced drivers and public transport vehicles to take another route. Additionally, each year, Ontario experiences a variety of hazardous weather events that close roads. For instance, just 3 months ago in January 2026, the snowstorm closed down main highways like Don Valley Parkway and a section of Highway 400 (The Weather Network). **The goal of this project is to create a tool that helps transportation people in charge analyze and understand how much a closure of particular swaths of road would disrupt commute and motorists in Ontario by shortest path graphical analysis**.

To be more specific, the analysis by this tool would be based on the extracting and displaying the next shortest paths (not just 1, but as many equivalent shortest paths) to get from the two points that the set of roads that closed down used to connect. As length of roads are usually with decimal precision, therefore, it is hopeless to expect that there would ever be two shortest paths to those two junctions. For instance, consider how a shortest path of 1.2 meters would be considered as not equivalent to a path of 1.3 meters to get from point A to point B, even though they both just differ by 0.1 meters. However, if we ignore the decimals, which we can do because their contribution to length of road is trivial to even pedestrians, just like in previous example where difference of a path not being a shortest path was just 0.1 meters. Hence, our analysis this way would give us a more realistic view of what paths people will choose to go from junction A to junction B, after the road has closed down between those two junctions. Highlighting the length of new shortest path side by side with length of previous shortest path before road closure would give an idea of the scale of impact and that too from different point of views. For instance, if the distance of shortest path goes from 100 meters to 1000 meters, that's a massive problem for pedestrians (but not for motorists). Lastly, actually highlighting the new shortest paths on map one by one would give the person in charge an idea of the structure of roads. This means if a if one, instead of going straight from junction A to junction B, now has to because of road closures take two left turns, then that would compound the time for not only the motorists who used to use that path, but also the motorists who have to now wait longer in queue as there are now other cars waiting for left turn on the signal. Ambulances who have to use that route will also take longer due to more cars using the same route and congesting at intersections. Therefore, showing different shortest paths side by side, previous shortest path length before closure vs after closure and ignoring small trivialities like decimals would give a great idea for the person responsible about the impacts of a future road closure. This would then help that person take proactive decisions to mitigate the effects of road closures.

The dataset to be used is the Ontario Road Network from Ontario Ministry of Natural Resources, available at https://geohub.lio.gov.on.ca/datasets/mnrf::ontario-road-network-orn-road-net-element/about (Ontario Ministry of Natural Resource). As of March 2, 2026 11:54 pm, this dataset contains 1,119,553 edges (roads) where a bidirectional road is counted as two different edges. It also contains 433,478 unique nodes. The following piece of JSON, with similar fields, whose example has been demonstrated already in it, and unneeded metadata in response being omitted for brevity, obtained by hitting one of paginated api https://ws.lioservices.lrc.gov.on.ca/arcgis2/rest/services/LIO_OPEN_DATA/LIO_Open09/MapServer/0/query?outFields=OGF_ID,FROM_JUNCTION_ID,TO_JUNCTION_ID,LENGTH,DIRECTION_OF_TRAFFIC_FLOW,OBJECTID&returnGeometry=true&outSR=4326&f=json&where=OBJECTID%20%3C%2047770000 is:


```
{
    ...
    "features": [
    ...

        {
          "attributes": {
            "OGF_ID": 1500457262,
            "FROM_JUNCTION_ID": 7221681,
            "TO_JUNCTION_ID": 7221677,
            "LENGTH": 434.953,
            "DIRECTION_OF_TRAFFIC_FLOW": "Negative",
            "OBJECTID": 47769490
          },
          "geometry": ...
        },
        {
          "attributes": {
            "OGF_ID": 1500452584,
            "FROM_JUNCTION_ID": 1500124587,
            "TO_JUNCTION_ID": 1500124588,
            "LENGTH": 52.924,
            "DIRECTION_OF_TRAFFIC_FLOW": "Both",
            "OBJECTID": 47769586
          },
          "geometry": {
            "paths": [
              [
                [-78.6984056, 45.0490474990568],
                [-78.6981341, 45.0494830990568]
              ]
            ]
          }
        },
        {
          "attributes": {
            "OGF_ID": 1500452564,
            "FROM_JUNCTION_ID": 6052867,
            "TO_JUNCTION_ID": 6052870,
            "LENGTH": 85.19,
            "DIRECTION_OF_TRAFFIC_FLOW": "Positive",
            "OBJECTID": 47769570
          },
          "geometry": ...
        }
        ...
```

```
    ]
}
```

Extracted from documentation of this API, the following information about this JSON is important to note (Ontario Ministry of Natural Resources 16, 59). The JSON here shows that road with object id 47769570 runs from vertex id 6052867 (FROM_JUNCTION_ID) to vertex id 6052870 (TO_JUNCTION_ID) only in the forward direction. In contrast, the road from vertex id 1500124587 to vertex id 1500124588 runs both ways, while the road from vertex id 7221681 to 7221677 only runs in reverse direction. The first element of attribute `paths` in `geometry` gives a list of coordinate tuples, that taken together give a real world polyline that connects these two vertices and gives the road its actual shape.

However, the API is paginated and only gives about 2000 records at most for one call. This means 500 API calls would be needed, along with a slight delay as well to prevent the API server from thinking that we are doing a DOS attack on it and hence blocking us. Therefore, we wrote code to hit this API with slight delay and with paginated cursors due to its 2000 roads limit, accumulate all data from it and save it to a file called `ontario_road_network.geojson` (the file shared). Also, we made sure that the url query is in a way that unnecessary data does not get returned: only the fields `OGD_ID, FROM_JUNCTION_ID, TO_JUNCTION_ID, LENGTH, GEOMETRY, OBJECTID and DIRECTION_OF_TRAFFIC_FLOW` were included in our api query and what we saved to the file. To prevent file from becoming huge in size, it does not have indented json. Also, it is in format geojson, which is just a type of json with strict structure imposed on it to make it suitable for geographical computations.

1 Computational Overview

The road network is represented as a weighted directed graph, where each vertex represents an intersection or road endpoint in Ontario, and each edge represents a road segment connecting two of those vertices with a weight representing the distance between them in meters. This graph structure directly models the road network because roads inherently connect two points, and removing an edge in the graph mirrors a real-world road/segment of road closure: allowing the analysis of which road path (next shortest path) that closure would lead the motorists to take to travel between the two vertices this segment used to connect. Graph also allows us to run Dijktra's algorithm (more on that later), thus allowing computation of shortest paths. Therefore, Graph is the backbone of our project, and it is represented by Graph class in graph.py file

However, Graph alone does not entirely suffice. Because we are computing multiple shortest paths of same length (ignoring decimal part) from a source junction to a target junction, and the usage of trees helps us store and keep track of these paths. The class PathTree, found in path_tree.py, is a tree class to represent multiple paths from the source node to the target node. The target node is the root of the tree, and the leaves are the source nodes, so each path is traced from a leaf (a source node) all the way up to the root (the target node) of the tree. The reason it is upside down is because in our version of Dijktra's, it is a lot easier and efficient to track each shortest path back from target vertex to source then to do it the other way. PathTree was indispensable for this job, because it is not just about multiple shortest paths to target, but rather a lot of times there are also multiple equivalent shortest paths to another vertex which lies along the shortest path, thus giving the task of tracing all these paths a branching structure. Tree is a natural choice for this representation of multiple paths therefore. However, we don't return a tree back to our other classes, as it is cumbersome to traverse a tree to get a shortest path to display. Rather, after running Dijkstra's algorithm to compute all the shortest paths from a source node to a target node, the returned object is an instance of the ShortestPathResult class in graph.py. This class is used to store all the possible paths from the source to the target node in its instance attribute, all_shortest_paths list. This list is populated by calling the function get_all_possible_paths method of class PathTree, which returns all possible paths after the PathTree tree has been built by Dijkstra algorithm in the form of a list of list by storing each traversal from depth point of view from root to leaf in a separate list. All paths in this list have the shortest length from the source to the target node, which is stored in the instance attribute of the ShortestPathResult class, length.

Overall, Graph class is the backbone of our project, but it cannot do its job without PathTree.

Types of computations to perform on the data:

1. Display a graph representation of road network on Ontario map.

2. Make road segment(s) removable by clicking on it/them, using a modified version of Dijkstra's algorithm to compute all shortest paths of equal minimum length between point A and point B that the removed road segment(s)

used to connect previously. Dijkstra's algorithm is used to give the shortest path from one source vertex to all other ones, and it has a time complexity of $\Theta(V + ELogV)$ where E is the number of edges (roads) and V is the number of vertices (intersections/road start/end point), so it is best suited to solve the problem efficiently for this data (GeeksforGeeks). 2. Make a road segment removable by clicking on it, and use Dijktra's algorithm to compute the length of shortest path between point A and point B that the removed single road segment use to connect previously. Dijktra's algorithm is used to give the shortest path from one source vertex to all other ones, and it has a time complexity of $\Theta(V + ELogV)$ where E is the number of edges (roads) and V is the number of vertices (intersections/road start/end point), so it is best suited to solve the problem efficiently for this data (GeeksforGeeks).

3. Using the same algorithm as above, build a collection of the shortest paths from the graph and highlight these shortest paths on the map in a distinct color. If no path exists, the program displays a message indicating those two nodes are now disconnected. 3. Using the same algorithm as above, build a collection of shortest paths from the graph and highlight these shortest paths on the map. In case of a tie, all shortest paths of equal length are shown. Show this shortest path on the map in a distinct color. If no path exists, the program displays a message indicating those two nodes are now disconnected.

4. Filter and display only the part of graph which is currently in view, as drawing all polylines at once will lead to performance issues. Roads are filtered based on two criteria: whether any coordinate of the road falls within the current map bounds, and whether the road's length exceeds a threshold derived from the current zoom level. At zoom levels below 13, only roads longer than this threshold are shown. At zoom level 13 and above, all roads within bounds are shown.

The following Python libraries are used in this project to accomplish its tasks. The **Shapely** library is used for click detection on the map. Specifically, the `MultiLineString` class represents a road's geometric polyline as a Shapely geometry object, and the `distance` method of the `Point` class computes the shortest distance between a user's click point and each visible road's polyline in `StreamlitManager._get_road_id_by_selection` in `streamlit_manager.py`. Since the `StreamlitManager._get_road_id_by_selection` is only called when some polyline is clicked, the job now is to figure out which line was clicked, which we do by computing distance to each polyline through shapely and returning the road associated with the polyline with least distance to click. The **orjson** library's `orjson.loads` function in `fast_geo_json_file_loader.py` parses the big road network geojson file efficiently into a Python dictionary (a lot faster than json library does), which is then used by `RoadManager.fetch_data_and_build_graph` method in `road_manager.py` to build the graph. The **Streamlit** library's `st.session_state` is used in `main.py` to persist the `StreamlitManager` instance across reruns. The **Streamlit-Folium** library's `st_folium` function renders the interactive map and captures user clicks, enabling bidirectional communication between the map and the app. Finally, the **Folium** library's `Map` class creates the base map of Ontario, and its `Polyline` class draws each road segment using the `geometry` attribute of each `Road` object.

A modified version of Dijkstra's algorithm is implemented in `Graph.compute_shortest_path` in `graph.py`. Unlike the standard algorithm which returns only one shortest path, this version collects all shortest paths of equal minimum length. The result is returned as a `ShortestPathResult` dataclass containing the shortest length and all equal shortest paths. If no path exists between the two nodes, `None` is returned to indicate disconnection.

Apart from the shortest path algorithm, the program performs additional computations on the data. To avoid performance issues from rendering over one million road segments at once, `StreamlitManager._update_visible_roads_by_bounds` in `streamlit_manager.py` filters roads based on the two criteria described in bullet point 4 above. The average road length, stored as `_road_average_length`, is computed during initialization and used as part of this threshold calculation.

The graph will be displayed using folium (a popular library to display detailed maps) and streamlit_folium library, where roads that connect vertices appear as polylines. The folium library's Map class is be used to create the map of Ontario and center it on screen, and then its GeoJson class is be used to create polylines from the geometric polyline data fetched from the api (see sample data), but folium library itself does not allow bidirectional communication (Story, "Getting Started — Folium"; "PolyLine — Folium"). To solve this issue, streamlit_folium library's st_folium function will be used to render the map created by folium, and then the callback provided to this function will be able to capture clicks on the map (Zwitch). This bidirectional communication capability is the reason why folium and streamlit_folium were chosen to represent the maps, as this is required for polyline removal and displaying shortest path interaction. Going back to the captured clicks, these clicks coordinates will then be passed to Shapely library's distance method of LineString class (which captures a single polyline point) to see if the click coordinates are on a polyline, and if yes, then the relevant road to which the polyline belonged to would be shown in different colour to highlight it being removed (Gillies). Shapely makes concentration on other important

features easier, as it handles click detection calculation to a degree. Then, custom written Dijktra's algorithm would be run to compute the shortest paths, and it would be highlighted on the map taking advantage of streamlit_folium's bidirectional communication capabilities due to how streamlit's architecture automatically reruns the app on each update. Finally, streamlit (on top of which streamlit_folium is built) library's text method is used to show the length of new path besides the map (Snowflake Inc., "Basic Concepts of Streamlit"; "St.markdown").

Instructions for Running the Program

To run this program, follow these steps:

1. Download the `ontario_road_network.geojson` file shared via one drive shared link- U of T One Drive Link Click Here (requires University of Toronto login), unzip it, and place `ontario_road_network.geojson` at the project's root directory. Plan B: In case if the link does not work, check the email csc111-2026-01@cs.toronto.edu for an email from elias.alhazzaa@mail.utoronto.ca

2. Ensure Python 3.13.5 is installed and install all required libraries using: `pip install -r requirements.txt` .

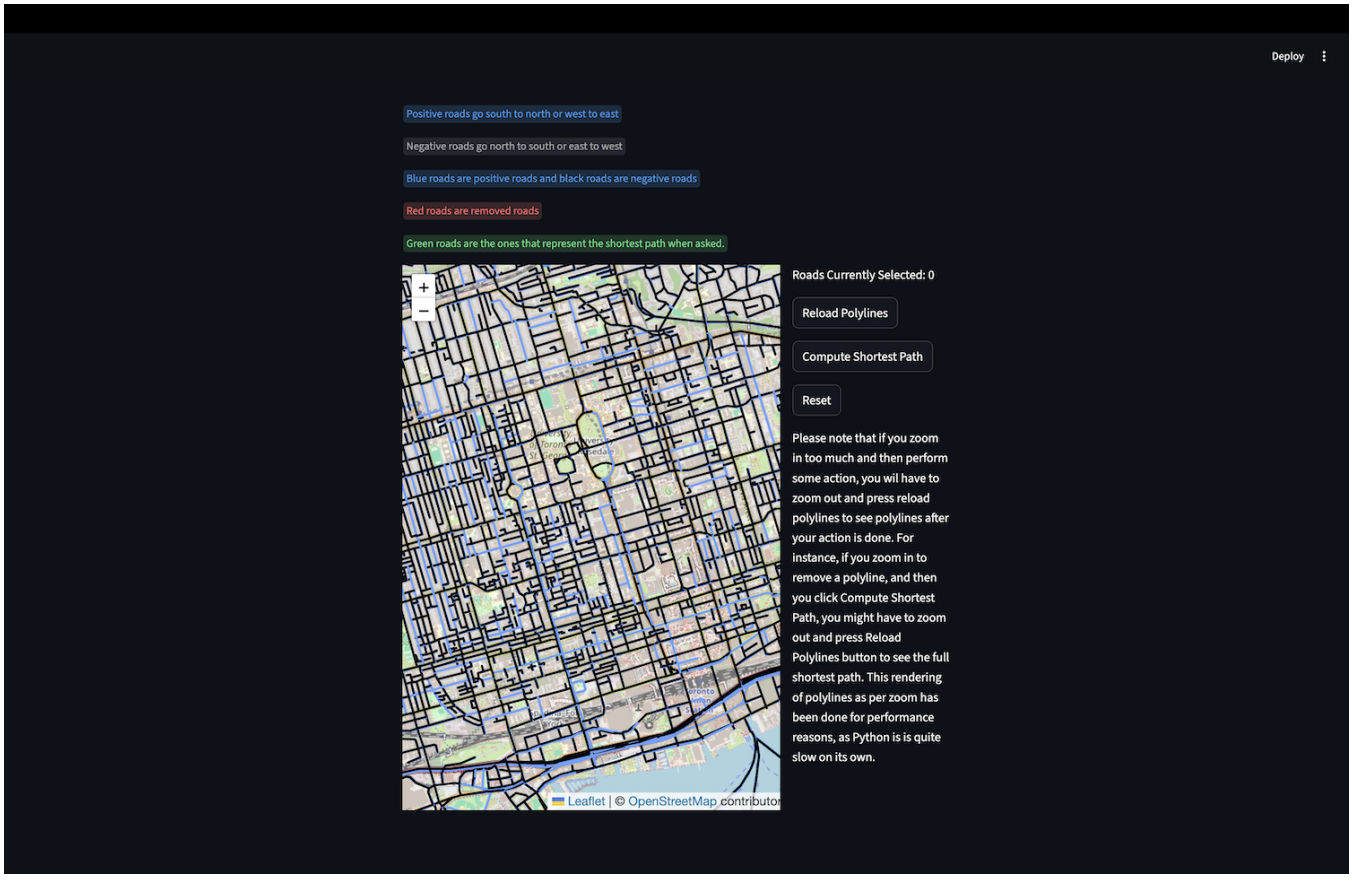
3. Open your terminal and run the following command from the root of the project: `streamlit run main.py` .

Important Note: If you are using Linux, don't run this project from **root directory of Linux**, as Streamlit does not work from there (Snowflake Inc.,"Basic Concepts of Streamlit"). Note: Do not attempt to run the program by clicking the "run" button in your IDE. Streamlit applications must be launched via the terminal command mentioned above to function correctly.

The next section describes what can be expected to be seen:

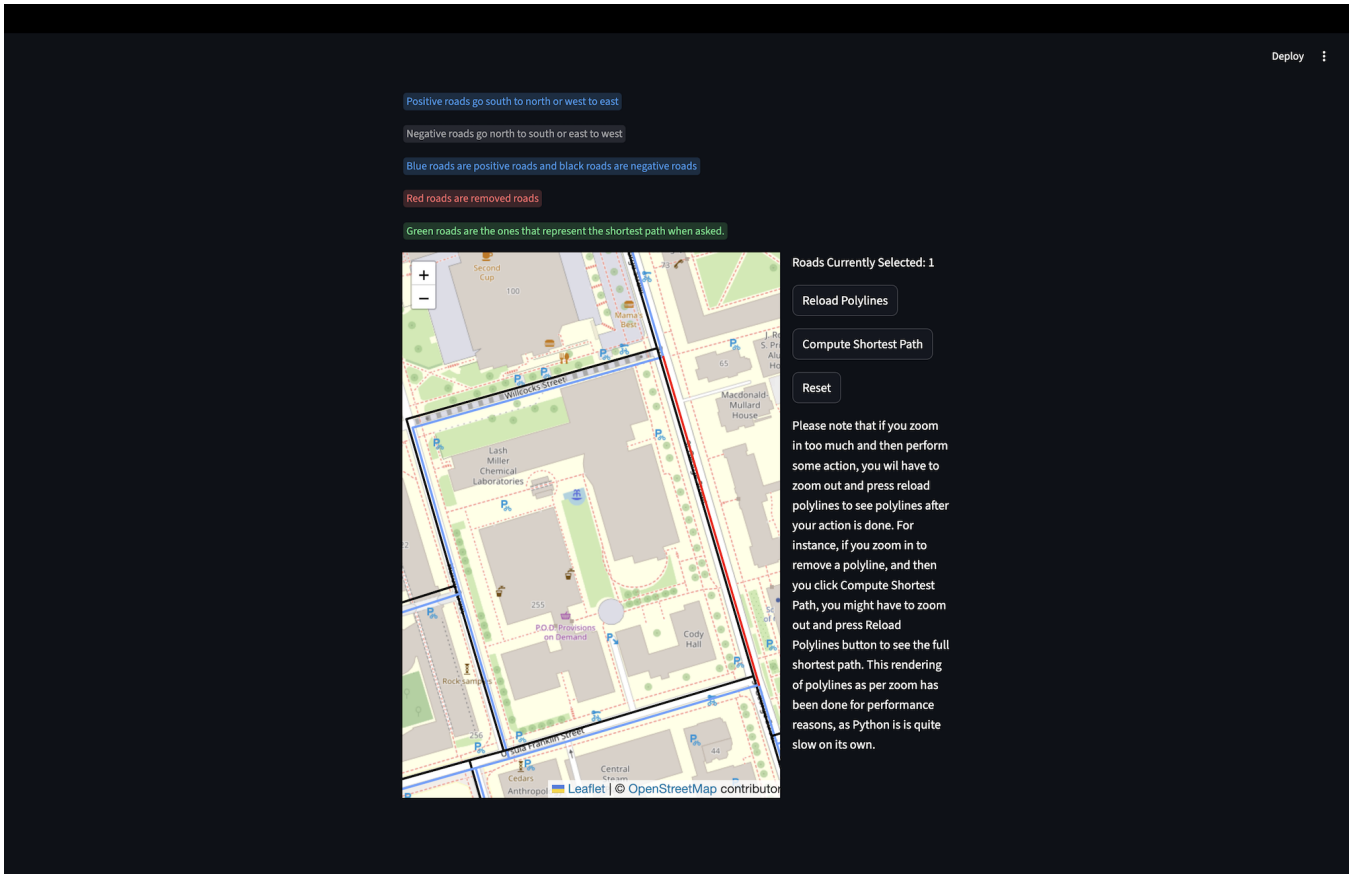
2 Results

The initial screen which opens up after starting the app will load data and map, and then it would show the map at initial settings as shown below. This is the University of Toronto area along with a lot of other surrounding places,

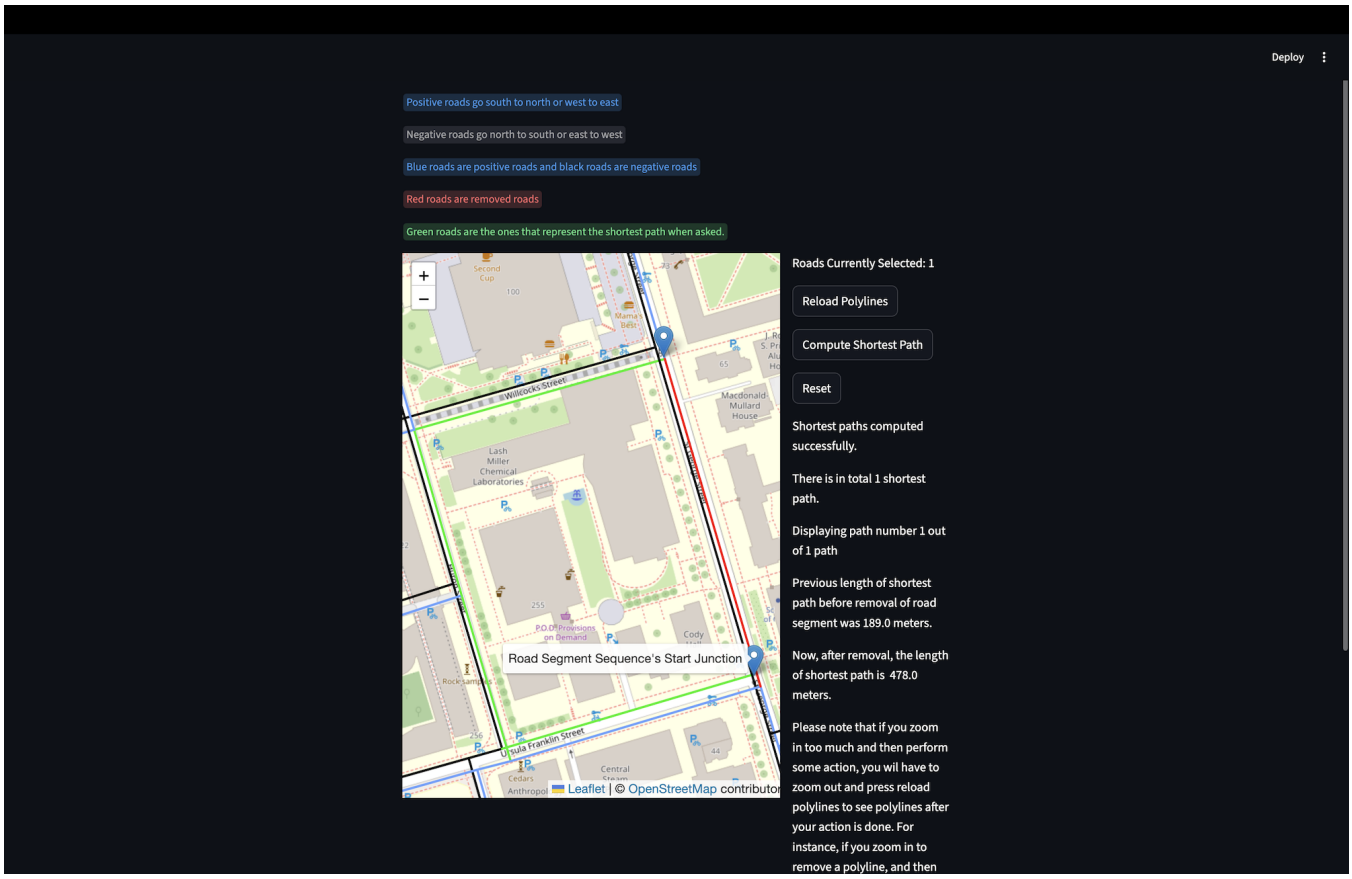


Now go to St. George Street near Willcocks and Ursula Franklin, and click on St. George Street going from south to north (the positive road) as shown below. After clicking it, it would turn red as shown in the image. Don't click on the one close to it: even though everything would work fine, but you would lose the track of this demo. You can also click reset button anytime you click something wrong.

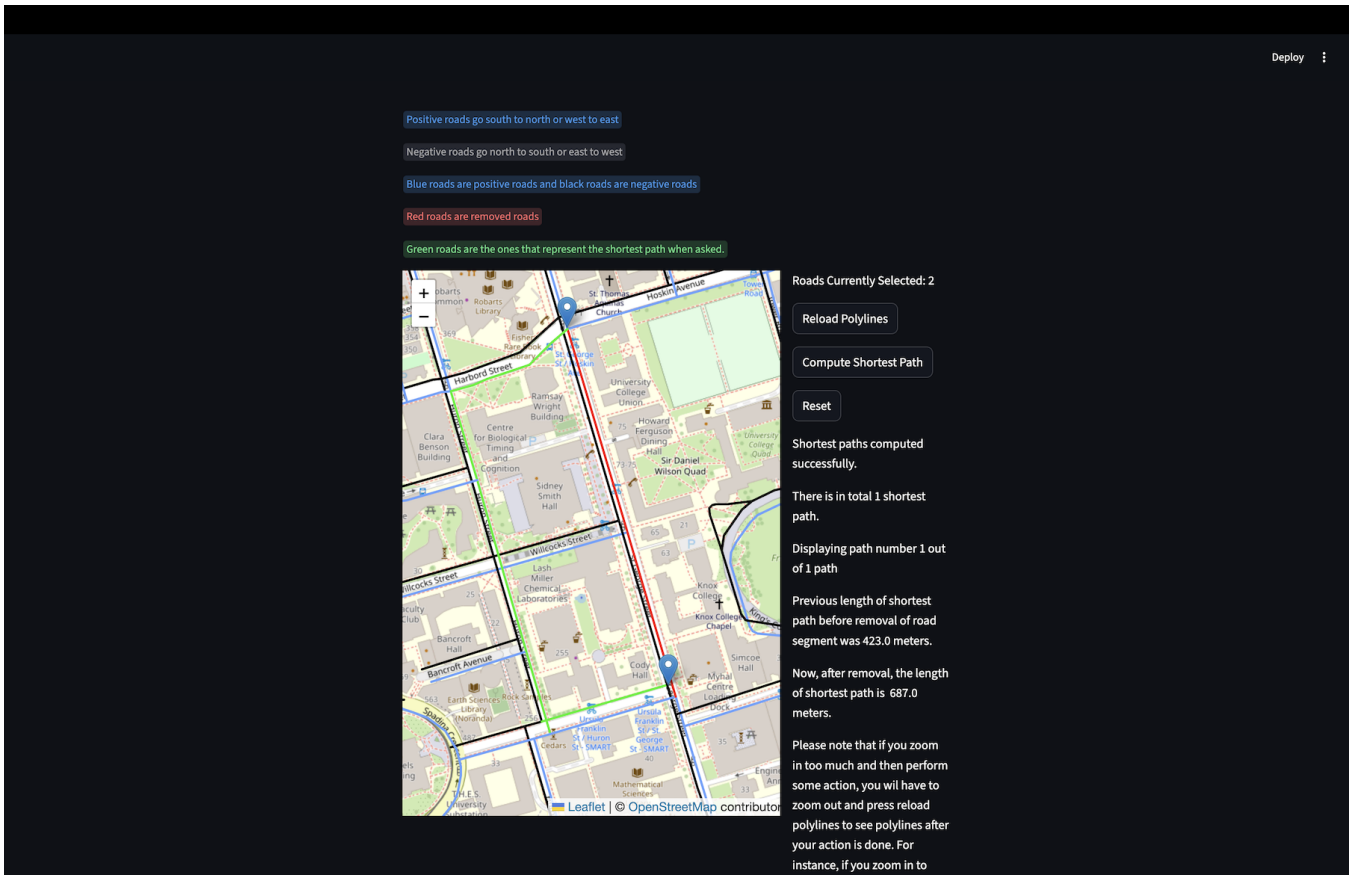
Note: Throughout this demo, you might feel as if some polylines have disappeared. This happens if because only polylines in your previous bounds and zoom are visible, and the new polylines don't get added unless you press the Reload Polylines button to reload them as per your new bounds and zoom level. This is intentional for performance reasons, otherwise frequent updates to map and slowness of Python would degrade performance a lot.



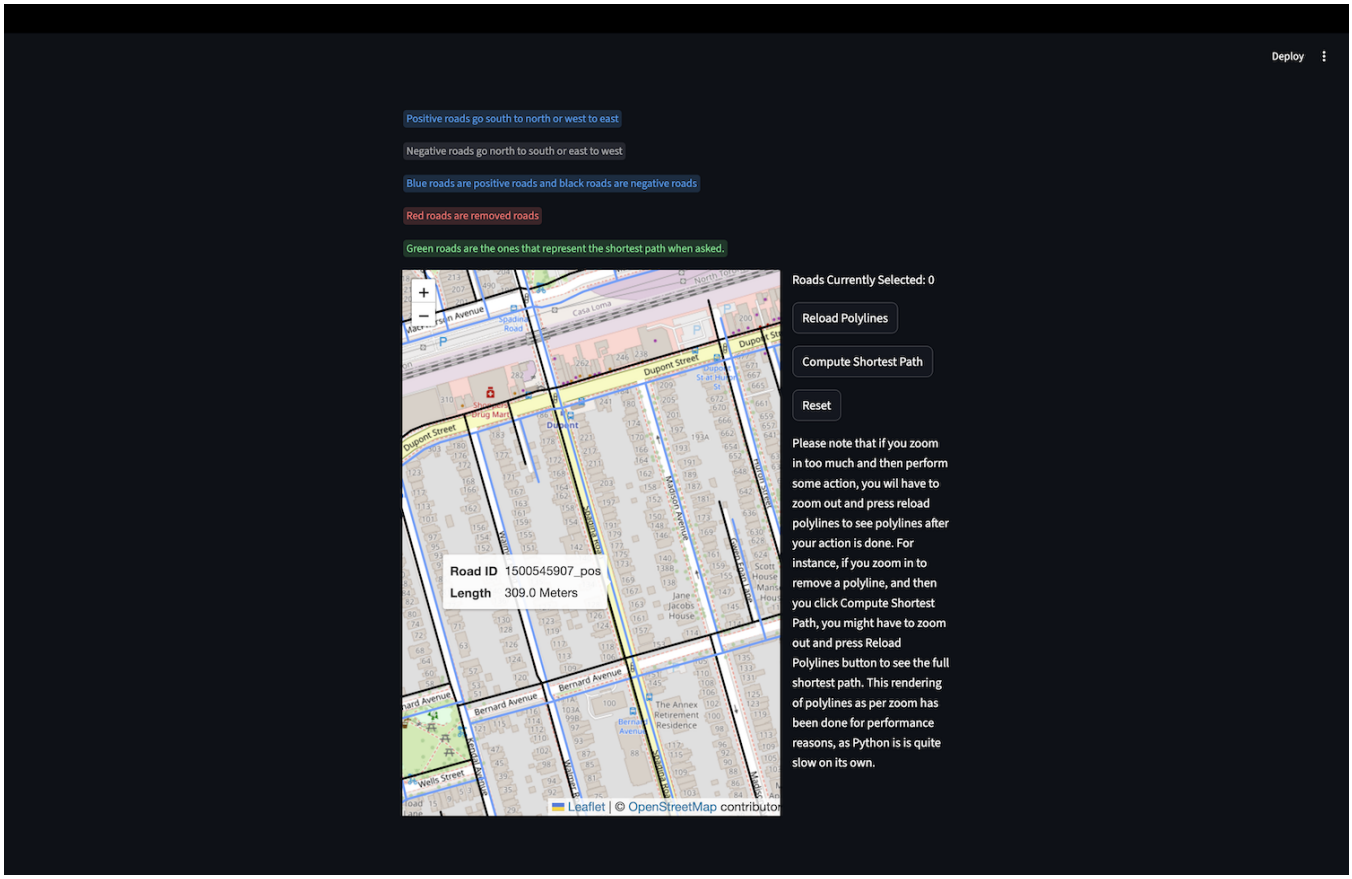
Press Compute Shortest Path button, and the shortest path between St. George at Ursula Franklin junction and St. George at Willcocks Street junction would get computed with the previously selected portion seen as deleted (this is simulating that road segment having a road closure). The shortest path computed would get displayed on the map in green colour, while the length of previous shortest path before closure and current shortest path after closure would be shown on the right. If you hover your mouse over the green road segments and add all the distances up, you will get the number on the right (but careful not to miss adding a road segment, a different road segment begins where the road meets another road, so in this case there are 4 road segments not 3, with the one on the left having two parts due to road meeting in the middle.



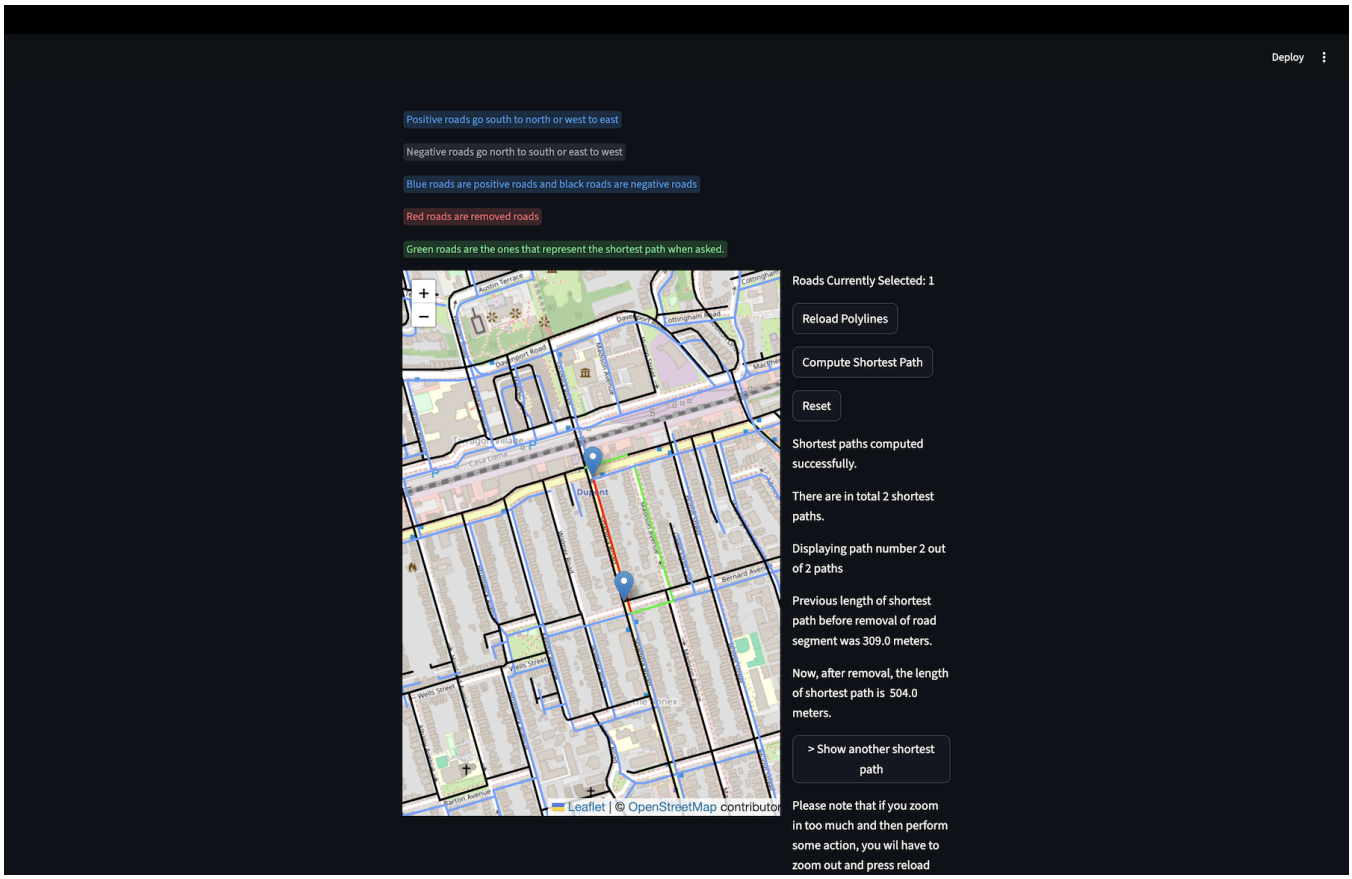
Now, we will see a demonstration of this with multiple path segments (major addition apart from multiple shortest paths). Click on St. George Street North side (positive road) again, but this time on the segment which goes from Willcocks to Harbord Street. It will get removed (meaning it will turn red in colour). After you it removed as per its colour, press Compute Shortest Path button, and zoom out if necessary. The green road segments all together form the shortest path from St. George at Ursula Franklin to St. George at Harbord Street after the road segments from from Ursula Franklin to Harbord at St, George street have been removed. The result should be similar to the image below depending on your zoom level and map position.

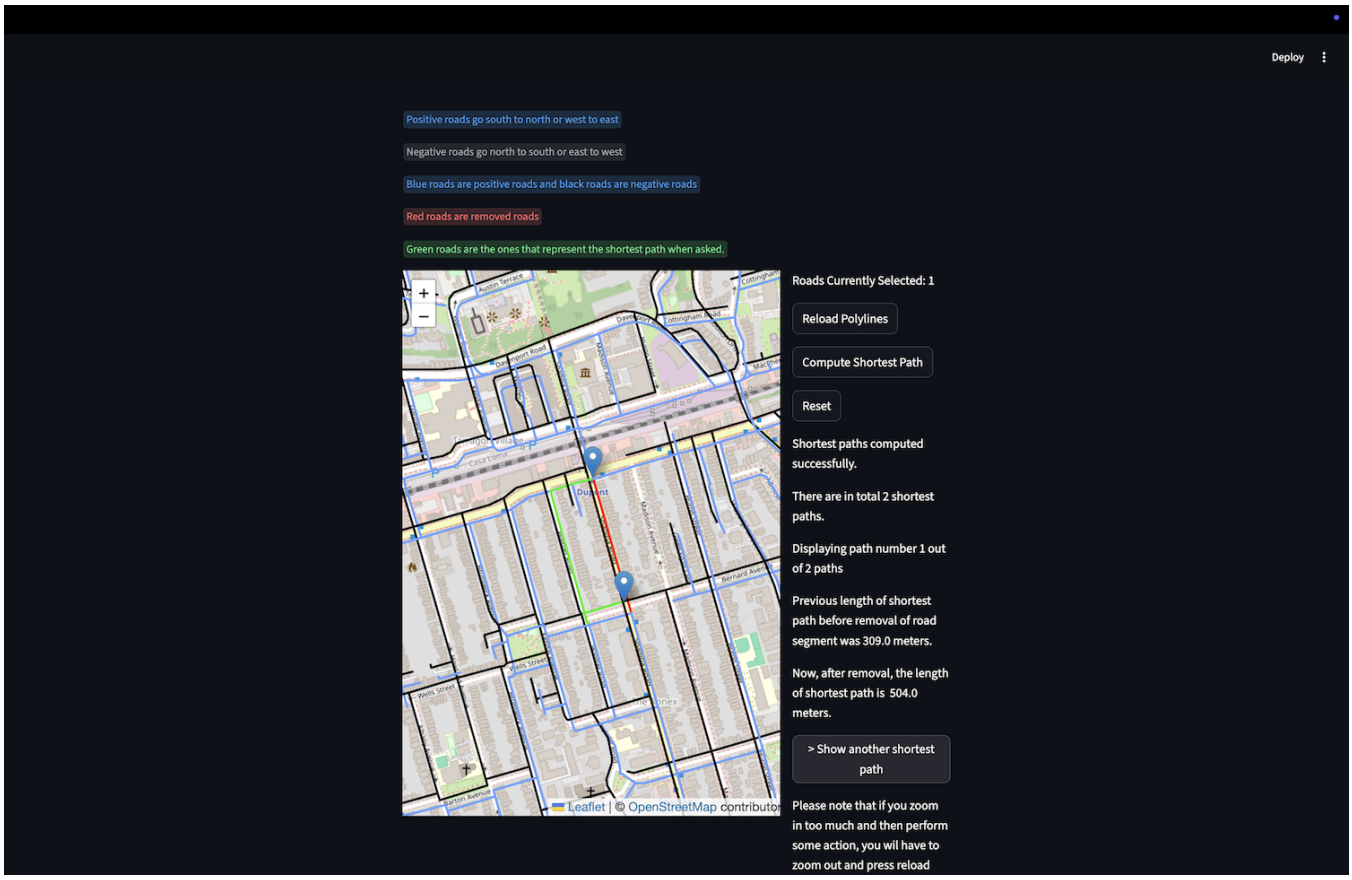


Press reset button to take every road to original state again. Now, we will see a demo for multiple shortest paths (our first major project enhancement). Scroll to Spadina at Dupont and press Reload Polylines if necessary. The image below shows that location.



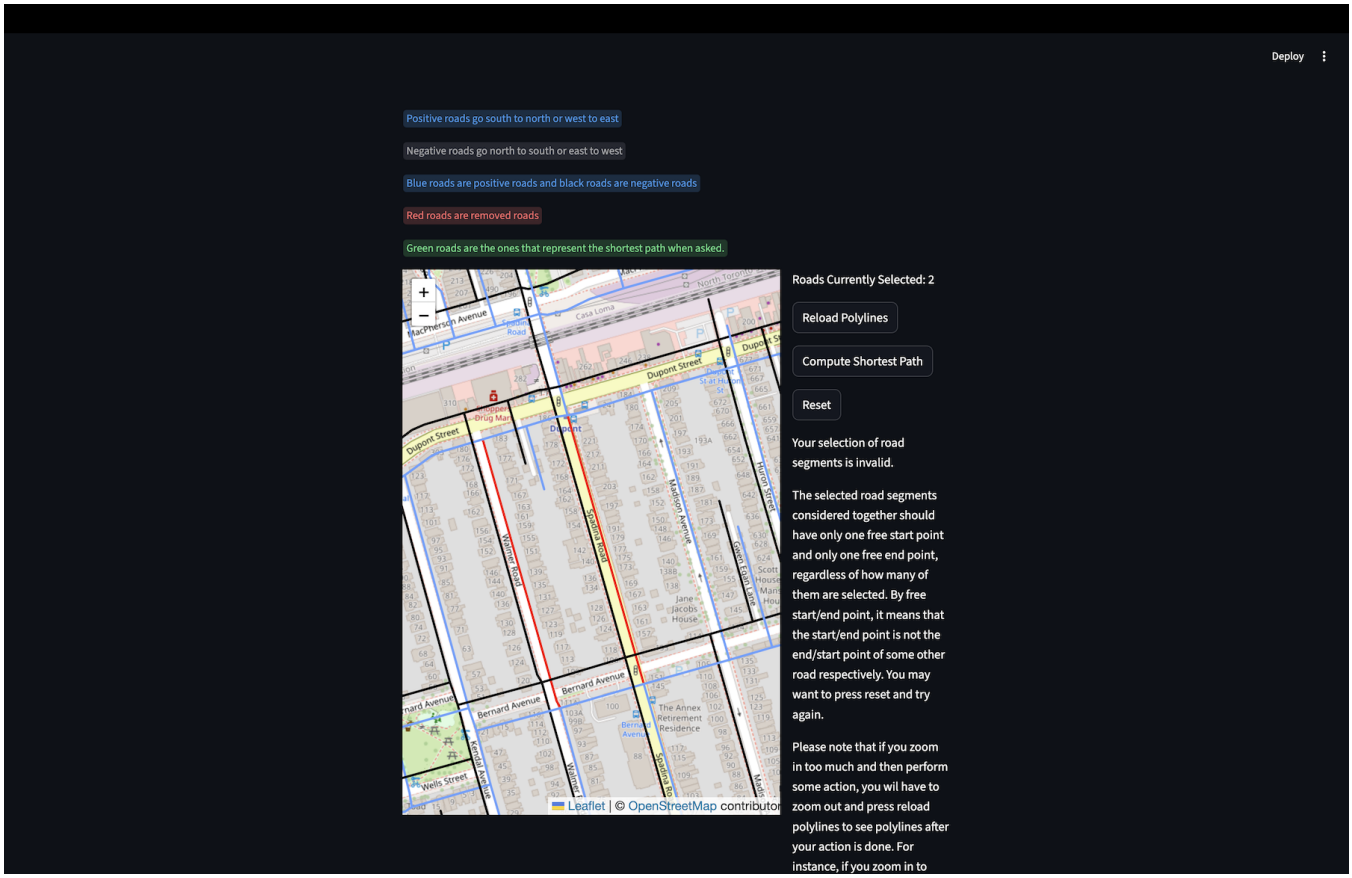
Click on Spadina Road that goes to Dupont Street (positive road) to remove it. Now click compute shortest path button. This will now show that there are two different equivalent shortest paths between the two junctions, but only one of them will get displayed to make it easy to follow. Click on Show Another Shortest Path, and the second shortest path will be displayed. Pressing the button furthermore will alternate between these two paths shown in the two images below (as there are only two equivalent shortest paths).



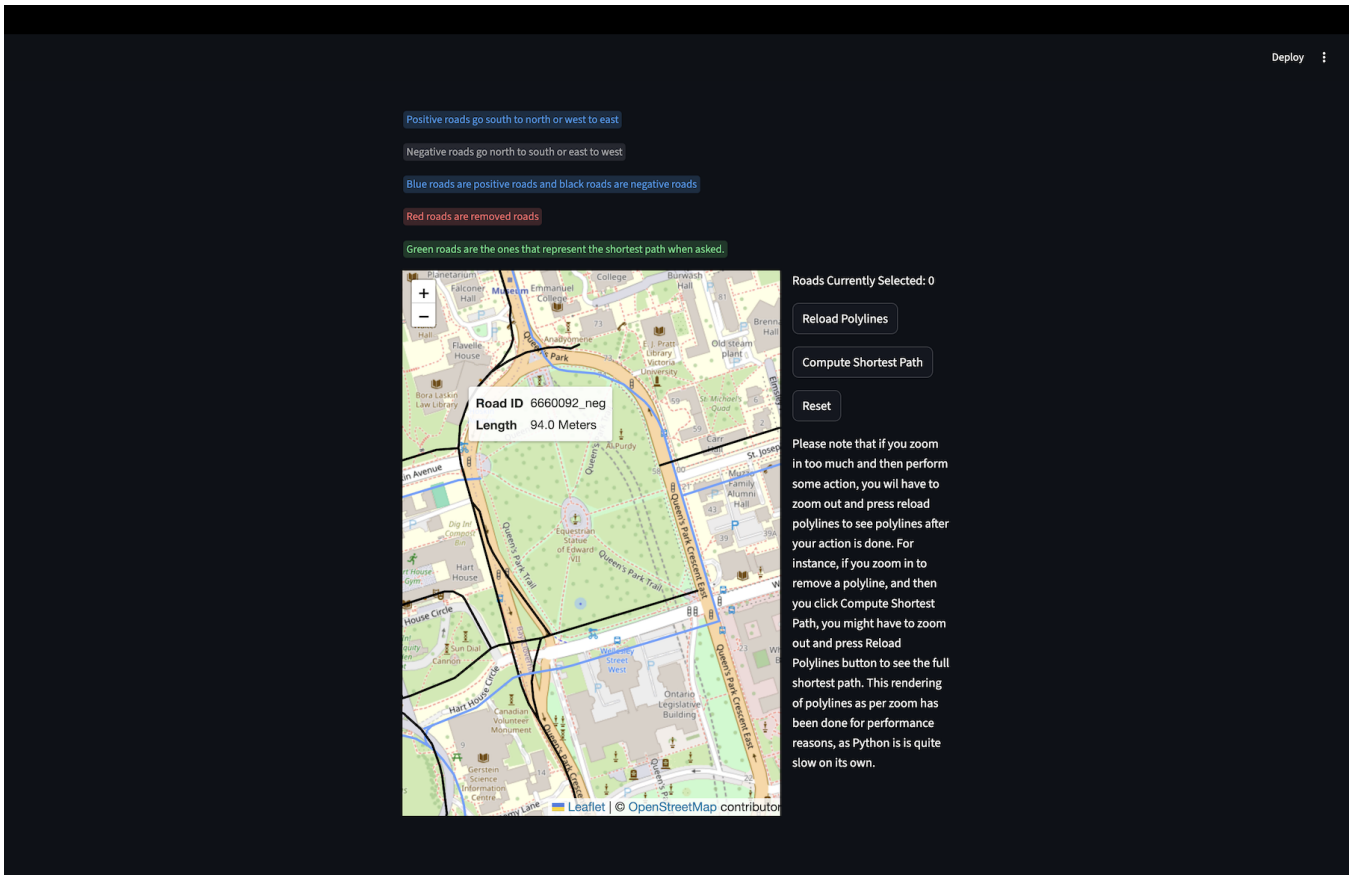


Press reset button. Now, we will explore our second last demo: the demo of invalid selection. Click on Spad-

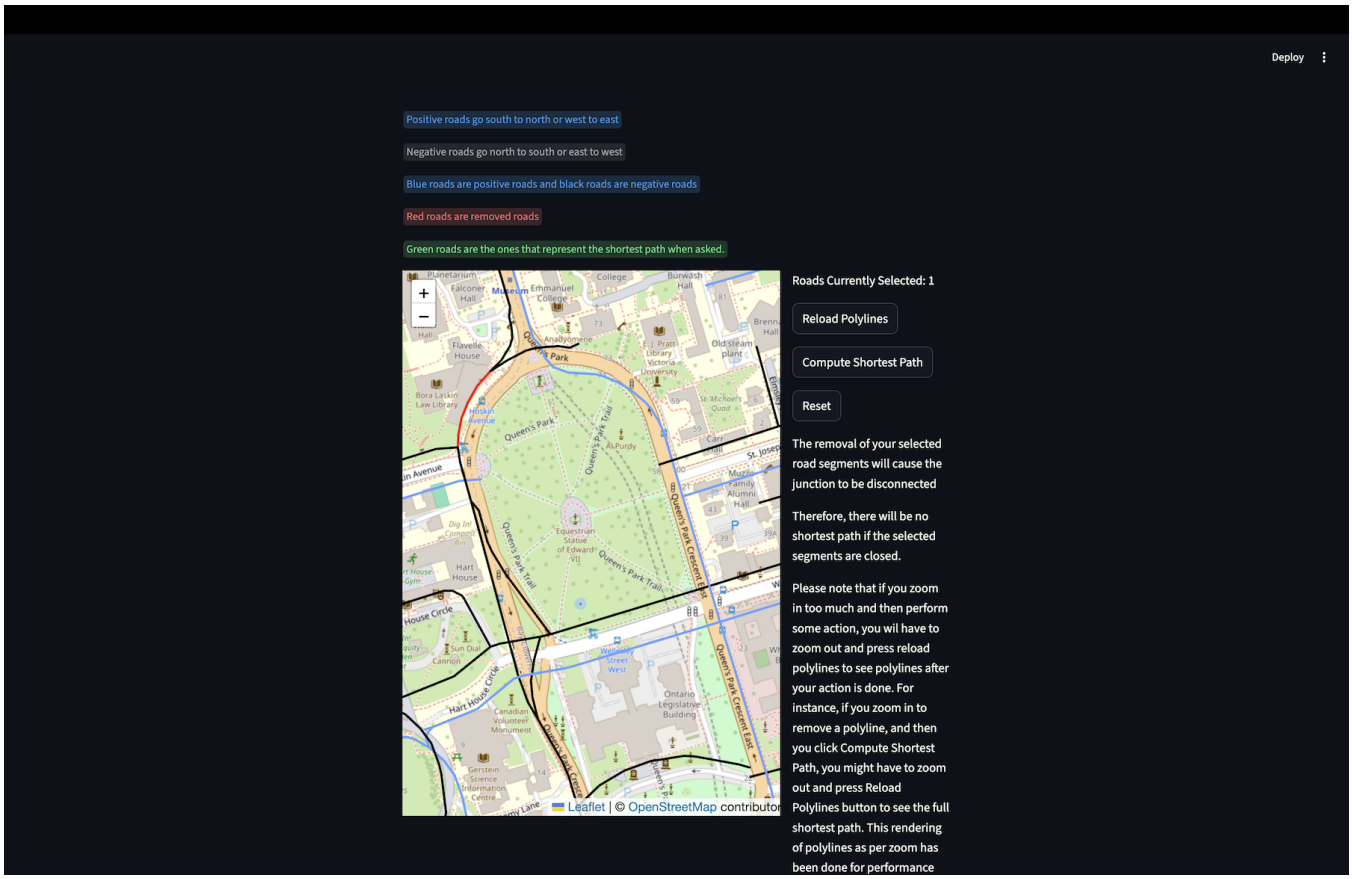
ina Road, and after it is removed, then click on Walmer Road which is parallel to it and does not join it directly. Then, click on Compute Shortest Path, but this time a shortest path will not come. This is because our Graph.check_is_valid_road_selection method will return None, as these we have two free start points and two free end points now violating our definition of validity. This means that we cannot decide the two vertices between which to run our Graph.compute_shortest_path method, and hence compute_shortest_path method is not invoked this time due to road sequence validity test failure.



Our last demo will be about what our app does when removing a road segment leads to junctions getting disconnected from each other. Scroll to Queen's Park area and click Reload Polylines if necessary. On the road id shown below in the tooltip (which comes up if one hovers over a road), click to remove it.



After the segment has been removed, press Compute Shortest Path method. The result would be that there is no shortest path, as removing the road leads to the two junctions getting disconnected. This can be seen directly from the map very easily as well. The red segment is connected only to black segments which are all negative roads, meaning they go from north to south in this case. Taking away the road path and no connectivity



This demo covered all features this app has: computing shortest path after removal of one road segment, after removal of multiple road segments, computing multiple shortest paths of same length, reset feature, lazy loading of polylines for performance reasons, alerting if junctions get disconnected after road removal and flagging invalid sequence of roads. Now, the reader is free to experiment as the reader of this report likes with any of these features wherever the reader wants on the map.

Changes from Proposal

For the final submission, we have created a better way to achieve the original proposed goal of building an app that allows an official to track how a road closure would affect traffic. We had two major changes to our original proposal in terms of the ways to better achieve that goal by incorporation of feedback.

The biggest change done was to use Dijktras to give all shortest paths, not just one shortest path. This required a study of the proof of Dijktras algorithm, coming up with own proof of a modified Dijktras algorithm we wrote where we had to show that when a vertex is added in all_shortest_paths_computed, all possible shortest paths to it have been computed already, and implementing it in O(V+E log V) upper bound time. Its performance was improved by using Python's heapq module's heap (which are also a type of tree) as priority queue, using list/tuples instead of dataclasses after emperically observing slowness of dataclasses, and finding a way to capture all shortest paths without passing around a copy of list which would also have slowed this method down a lot. Instead, a reference to previous vertex from which shortest distance came was kept in distance dictionary (in Graph.compute_shortest_path method) along with the road id it came from (as one vertex might have multiple roads to current vertex), and then using PathTree (a tree) to model path back from target to source where target was the root and were the leaves (discussed the why in more detail before), and then constructing a list of list using this tree to give back a list of shortest paths. As discussed in introduction section, showing multiple paths allows the person in charge to get a relatively better view of how people will use roads. This algorithm also ignores decimals due to what was mentioned in introduction section.

Our second major change from the proposal of our project was that instead of removing one road segment and finding any shortest path from the start point to the endpoint, we now give the user the choice to remove multiple segments. If the user chooses to compute shortest path, then in `Graph.check_is_valid_road_selection` method, we

check if sequence of roads is valid or not with a custom mini algorithm. We check validity by checking that there is at least one free start point (free meaning the start point is not the end point of another road) and at least one free end point (free meaning in this case that the end point is not the start point of another Road), and if there are more than one of roads with free start points and/or free end points, then we check if all such roads have the same start/end point respectively. This algorithm, if it decides the selection of roads as 'valid,' gives us a clean start junction id and end junction id between which we can compute the shortest distance. This algorithm allows the users to select multiple road segments, but in a manner that the selected road segments are not arbitrarily floating everywhere, hence making the idea of shortest path computation between two points meaningless. Since there was the possibility of by mistake selecting 'invalid' paths, we also created a reset button. Before this, Dijktras algorithm would have run after the road segment was actually removed from the graph, but now, it would have to be soft deleted and not hard deleted to allow for a reset (a reset would include all deleted roads being restored, see StreamlistManager.reset method and RoadManager.restore_removed_roads and Graph.restore_removed_roads that deal exactly with this). Hence, this change also forced us indirectly to add soft deletion for road (which is a directed weighted edge), and then to modify Dijktras in a way that it ignores the soft deleted ones. This was not possible without actually understanding how the algorithm works, and hence this compounded the complexity of soft deletion.

3 Discussion

The goal to create an app that allows a transportation person in charge to analyze how road closure affects traffic has been achieved to a degree. For road segments like the one near Queen's Park in our demo section, removing/closing which leads to the two junctions that the road segment used to connected getting disconnected, this program helps the person in charge immediately comprehend the vitality of that route, and hence that individual can then take steps to mitigate the closure to lessen its deep effects on the traffic where they cannot reach their destinations if that lies on one of those junctions.

For routes that have one or more shortest paths available, but the shortest path has more turns than the shortest path when original route segment was part of the road network, this means that not just the distance of the road would slow down the driver, but the turns would slow then the driver even more for the same distance and it would slow down other users of the road as well due to intersections swelled with more turning vehicles. However, despite this helping to achieve the goal to a degree, it does not give the complete picture. The complete picture would have been given by using Google Map's APIs to calculate average congestion and roads, and then modelling how the congestion now would increase over the roads above average congestion that a road segment is closed. This is because if the closure of shortest path leads users to use a road that was not used a lot previously anyways, this would mean that traffic flow would not have as much friction as anticipated by the map.

Another way using Google Map's APIs to use average time at specific time of day instead of road distance to determine which road users would take is from a motorist centric point of view. Motorists don't take roads that are the shortest in distance, and rather take roads which are shortest in time which the API would have helped in achieving. Thus, a Dijktras over average time a road segment takes from junction A to junction B would have given shortest path that the user is likely to take.

It can therefore be said that users who don't rely on apps like Google Maps that prioritize time over distance, and rather rely over the length of the road they take, then this analysis covers their behaviour well.

References

Works Cited:

GeeksforGeeks. "Dijkstra's Algorithm to Find Shortest Paths from a Source to All." GeeksforGeeks, 25 Nov. 2012, www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/.

Gillies, Sean. "Shapely.MultiLineString — Shapely 2.1.2 Documentation." Readthedocs.io, 2025, `shapely.readthedocs.io/en/stable/reference/shapely.MultiLineString.html`. Accessed 3 Mar.

2026.

Legislative Assembly of Ontario. "More about Ontario — Legislative Assembly of Ontario." Ola.org, 2024, www.ola.org/en/visit-learn/teach-learn-play/about-ontario/more-about-ontario.

Ontario Ministry of Natural Resources. "Ontario Road Network (ORN) Road Net Element." *Gov.on.ca*, 2020, geohub.lio.gov.on.ca/datasets/mnrf::ontario-road-network-orn-road-net-element/about.

—. ORN Road Net Element - User Guide-EN. 2025, Ontario Road Network - Road Net Element - User Guide (Word).

Snowflake Inc. "Basic Concepts of Streamlit - Streamlit Docs." Streamlit.io, 2026, `docs.streamlit.io/get-started/fundamentals/main-concepts|`.

—. "St.markdown - Streamlit Docs." Streamlit.io, 2024, `docs.streamlit.io/develop/api-reference/text/st.markdown`.

Story, Rob. "PolyLine — Folium 0.1.Dev1+G3ccc47c Documentation." Python-Visualization.github.io, `python-visualization.github.io/folium/latest/user_guide/vector_layers/polyline.html`.

—. "Getting Started — Folium 0.1.Dev1+G9ab63f7 Documentation." Python-Visualization.github.io, `python-visualization.github.io/folium/latest/getting_started.html`.

The Weather Network. "BURIED: Southern Ontario Digs out of High-Impact Snowstorm." The Weather Network, 15 Jan. 2026, www.theweathernetwork.com/en/news/weather/severe/buried-major-snowstorm-brings-road-closures-flight-cancellations-dvp-closed. Accessed 4 Mar. 2026.

Zwitch, Randy. "Callbacks." Streamlit.app, 7 July 2023, `folium.streamlit.app/callbacks`. Accessed 3 Mar. 2026.